

Lekcja 06 - Python w Data Engineeringu

2026-08-14

Data Engineering Course

Python w Data Engineeringu

```
extract -> Pydantic validation -> dataclass  
records -> transform -> load
```

Dzis przechodzimy od składni Pythona do małego pipeline'u.

■ Po co ta lekcja

Na rozmowie nie pytają tylko o petle.

Często pytają:

Jak pobierasz dane?
Jak sprawdzasz, czy payload ma sens?
Jak dzielisz kod na kroki?
Jak testujesz bez prawdziwego API?
Co zrobisz, gdy rekord jest błędny?

```
extract
  pobierz dane z pliku/API/DB

validate
  Pydantic sprawdza pola, typy, status i kwote

domain record
  dataclass Order reprezentuje poprawny rekord

transform
  zamien typy i policz wynik

load
  zapisz output
```

Kazdy krok ma inna odpowiedzialnosc.

■ Przykład z lekcji

Dwa wejścia:

```
orders.csv  
api_orders.json
```

Jeden output:

```
output/pipeline_output.json
```

Nie potrzebujemy prawdziwego API, żeby nauczyć się struktury pipeline'u.

■ Extract

```
def read_orders_csv(path: Path) -> list[dict[str, Any]]:  
    with path.open(newline="") as file:  
        return list(csv.DictReader(file))
```

Extract ma pobrac dane.

Nie licz tu revenue.

■ Validate: required fields

```
REQUIRED_FIELDS = {"order_id", "status", "total_amount"}

has_required_fields = all(record.get(field) for field in REQUIRED_FIELDS)
```

Pytanie:

Czy rekord bez order_id moze wejsc do transformacji?

Nie powinien.

■ Validate: Pydantic

```
class OrderPayload(BaseModel):  
    order_id: str  
    status: str  
    total_amount: float = Field(ge=0)  
    source: str = "unknown"
```

Pydantic sprawdza raw `dict` z pliku albo API.

Tu odrzucasz:

```
brak order_id  
ujemny total_amount  
tekst zamiast liczby  
nieznany status
```

■ dataclass Order

```
@dataclass(frozen=True)
class Order:
    order_id: str
    status: str
    total_amount: float
    source: str
```

Po walidacji nie nosimy dalej surowego `dict`.

Transformacje pracują na jasnym typie: `Order`.

■ Transform

```
def calculate_completed_revenue(records:  
list[Order]) -> float:  
    revenue = 0.0  
    for record in records:  
        if record.is_completed:  
            revenue = revenue + record.total_amount  
    return revenue
```

Tu liczysz metrykę na rekordach, które przeszły kontrakt.

■ Rejected records

```
@dataclass(frozen=True)
class RejectedRecord:
    record: dict[str, Any]
    reason: str
```

Nie wystarczy powiedzieć: "rekord był zły".

Chcesz wiedzieć:

```
który rekord odpadł
dlaczego odpadł
```

```
def write_json_output(payload: dict[str, Any],  
path: Path) -> None:  
    path.parent.mkdir(parents=True, exist_ok=True)  
    path.write_text(json.dumps(payload, indent=2))
```

Load zapisuje wynik.

Dzięki temu można go pokazać reviewerowi albo downstream userowi.

Pytanie:

Jak testujesz pipeline, który normalnie czyta API?

Odpowiedz:

Oddzielam funkcje transformacji od IO.
API payload zapisuje jako maly JSON fixture.
Testuje walidacje i transformacje bez prawdziwej
uslugi.
Osobno testuje, ze kod obsluguje blad HTTP albo
pusty payload.

Oddajesz:

```
pipeline_workflow.py  
test_pipeline_workflow.py  
orders.csv  
api_orders.json  
pipeline_output.json  
notes.md  
interview_answer.md
```

Najwazniejszy test:

```
Pydantic odrzuca bledny rekord,  
dataclass Order reprezentuje poprawny rekord,  
revenue liczy tylko accepted completed orders
```

■ Closing check

Powiedz na głos:

```
Extract pobiera dane.  
Validate sprawdza kształt przez Pydantic.  
dataclass Order opisuje poprawny rekord.  
Transform liczy metryki na accepted orders.  
Load zapisuje wynik.  
Testy powinny pokrywac bledny rekord i happy path.
```